An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

October 2020
Alexey Dosovitskiy,
Lucas Beyer,
Alexander Kolesnikov,
Dirk Weissenborn,
Xiaohua Zhai,
Thomas Unterthiner,
Mostafa Dehghani,
Matthias Minderer,
Georg Heigold,
Sylvain Gelly,
Jakob Uszkoreit,
Neil Houlsby

Outline

Transformer Architecture

How Multi-Head Attention works in Transformers, using Sebastian Raschka's book *Build a Large Language Model (From Scratch)*.

Vision Transformer Implementation

Previous attempts to utilize attention in computer vision, and the approach taken with ViT.

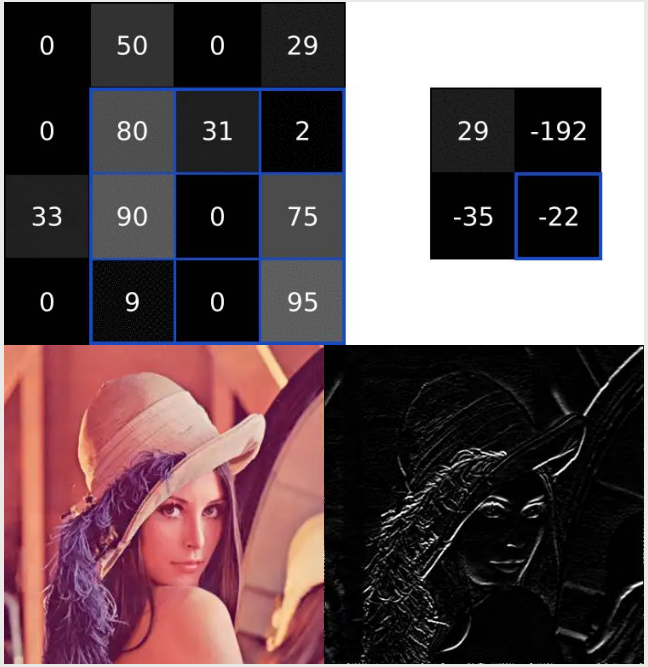
Performance versus Previous SotA

Evaluation of the performance of several ViT implementations and previous state-of-the-art models, compared against various benchmarks.

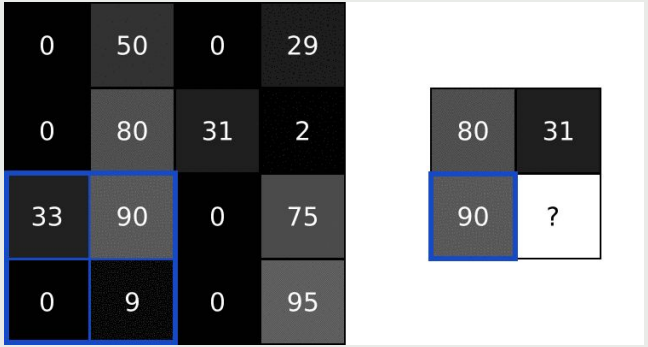
Background:
Convolutional Neural Networks

Credit to Victor Zhou *https://victorzhou.com/blog/intro-to-cnns-part-1*

Feature Detection



Pooling



Transformer Architecture: A Simple Attention Mechanism

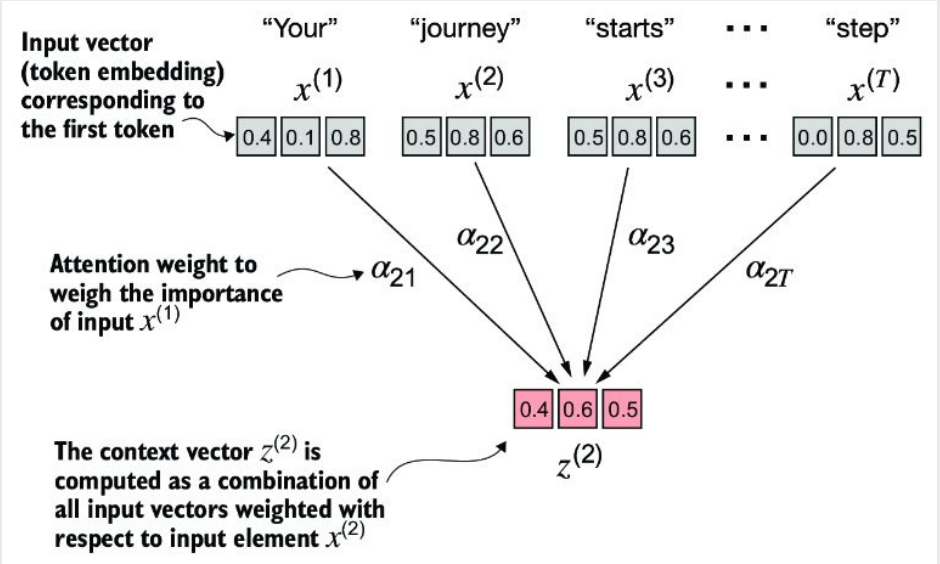
Credit to Sebastian Raschka

The transformer takes as input a series of **embedding vectors**, which are essentially arrays of numbers which represent some information. As an example, we'll be using this sentence as input:

"Your journey starts with one step"

```
inputs = torch.tensor(
  [[0.43, 0.15, 0.89],      # Your          (x^1)
   [0.55, 0.87, 0.66],      # journey       (x^2)
   [0.57, 0.85, 0.64],      # starts        (x^3)
   [0.22, 0.58, 0.33],      # with          (x^4)
   [0.77, 0.25, 0.10],      # one           (x^5)
   [0.05, 0.80, 0.55]]      # step          (x^6)
)
```

For each input (or **query**) in a series, an attention mechanism computes a **context vector**. This context vector is a combination of all other values in the series, each weighted by their relevance to the query.



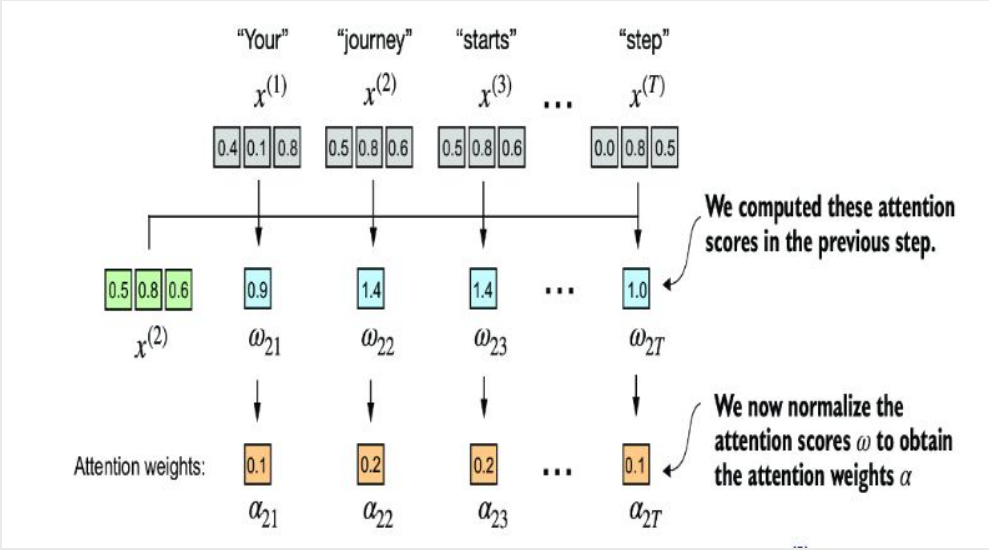
Transformer Architecture: A Simple Attention Mechanism

Credit to Sebastian Raschka

The **attention score** of an embedding vector with respect to the query is the **dot product** of the query and the embedding vector.

Query “journey”		x^1 “Your”		
0.55	x	0.43	=	0.24
0.87	x	0.15	=	0.13
0.66	x	0.89	=	0.59
				0.95

The attention scores are then normalized to get the **attention weights**.

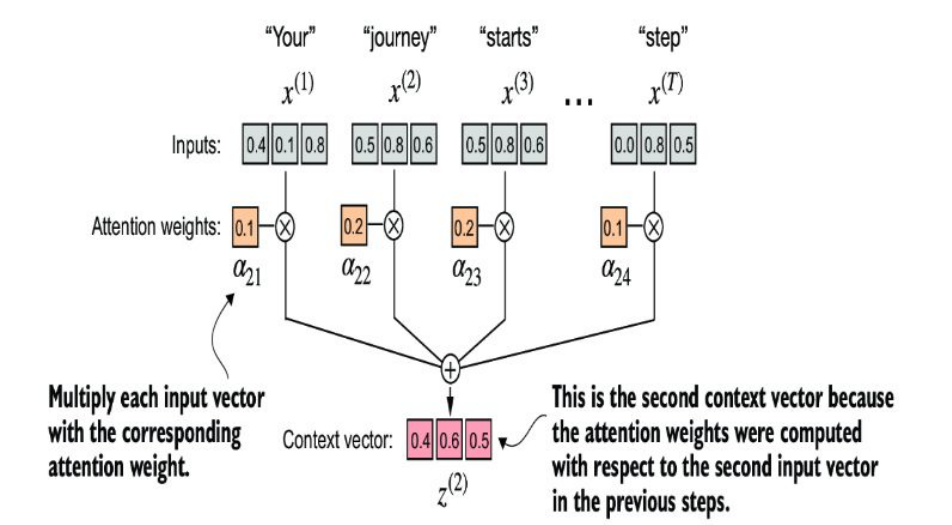


Transformer Architecture: A Simple Attention Mechanism

Credit to Sebastian Raschka

Now that we have the attention weights, we can compute the context vector for the query. Simply multiply each input vector by its attention weight, and then sum them to get the context vector.

Thus the context vector for x^2 (“journey”) is [0.4, 0.6, 0.5].



	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

Transformer Architecture: Attention With Trainable Weights

Credit to Sebastian Raschka

In practice, transformer attention uses trainable weights. Specifically, there are three trainable weight matrices for each attention head:

Wq → query weights

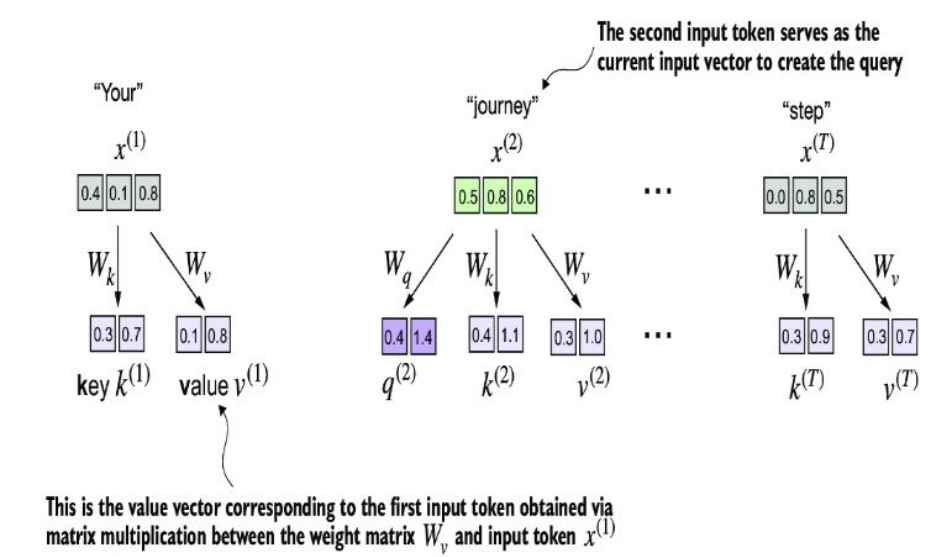
Wk → key weights

Wv → value weights

For each input, we calculate a **key vector** and a **value vector**. Additionally, for the query, we calculate a **query vector**.

These are calculated by multiplying the input by the corresponding weight matrix. For example:

```
query_2 = x_2 @ W_query
key_2 = x_2 @ W_key
value_2 = x_2 @ W_value
```



Transformer Architecture: Attention With Trainable Weights

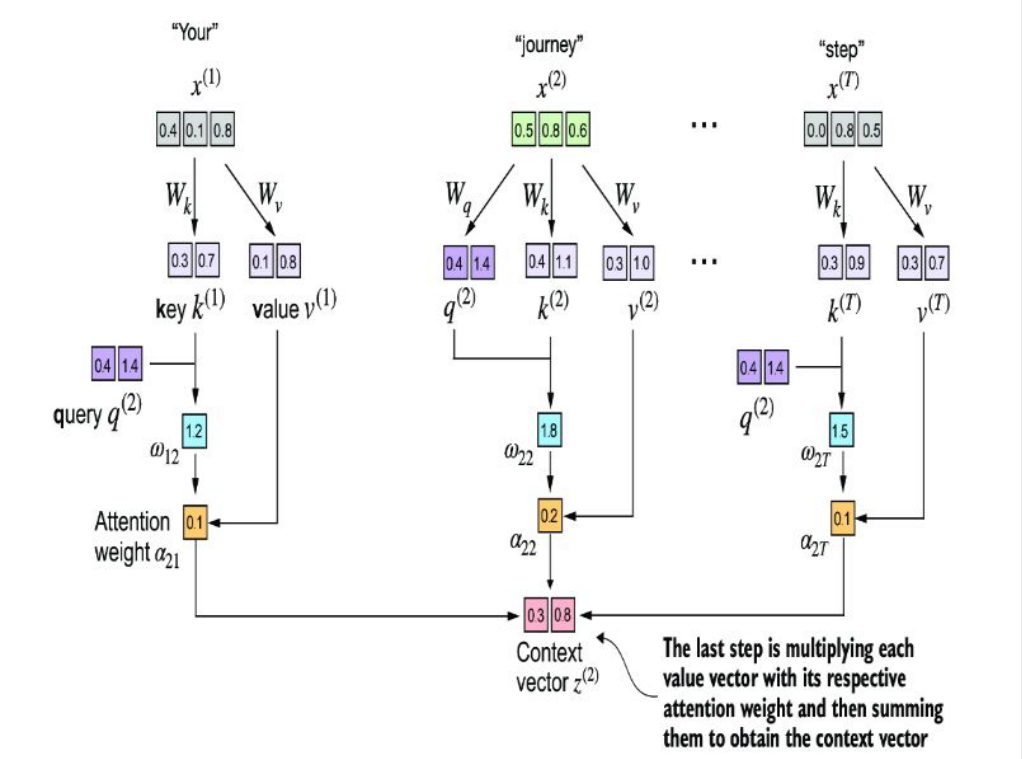
Credit to Sebastian Raschka

Now that we have key and value vectors for each input (and one query vector), we get the attention scores by multiplying the keys by the query vector.

These are then normalized with softmax to get the attention weights.

Finally, the context vector is calculated by multiplying the value vectors by the attention weights and summing the results.

At every step, some trainable weight is involved.

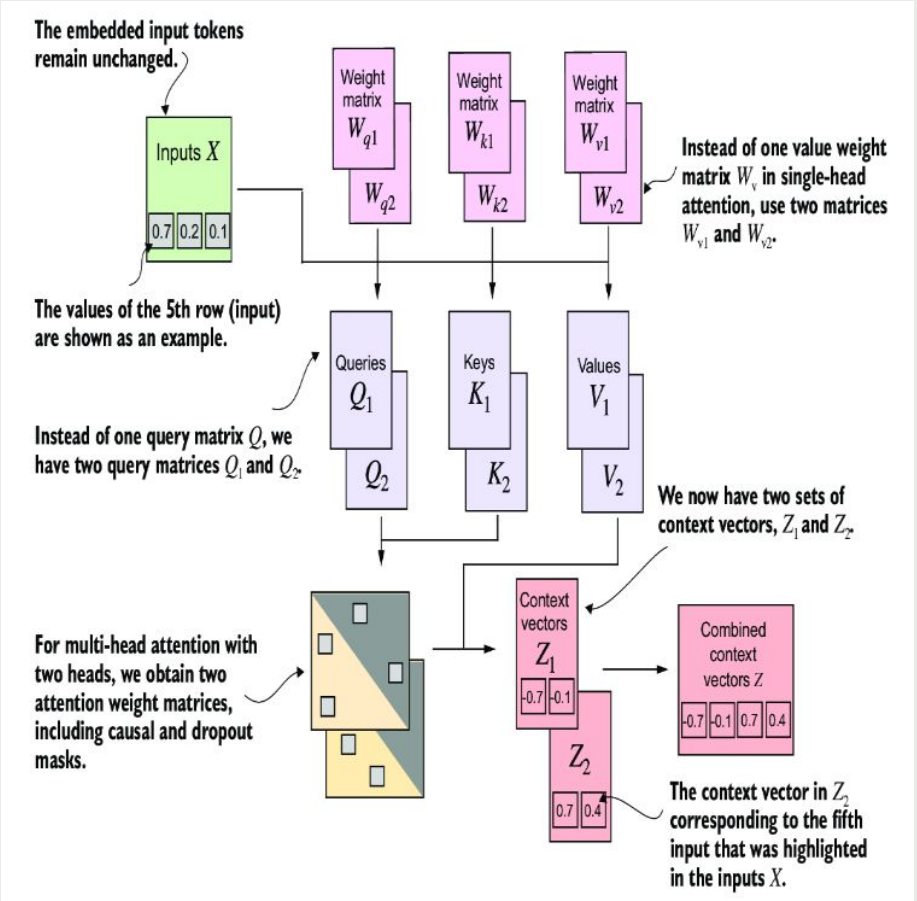


Transformer Architecture: Multi-Head Attention

Credit to Sebastian Raschka

Multi-head attention just means doing what we described multiple times and combining the output. Since weights are initialized with random values, each head will produce slightly different output, which may then move in different directions during training.

Note that multi-head attention wouldn't actually be implemented with unique query, key, and value matrices per head. The matrices are shared and scaled up by the number of heads, to reduce the number of matrix multiplication operations. Mathematically it has the same result.



ViT Implementation: What's the Problem?

Attention is computationally expensive for large images. If each pixel attends to each other pixel, training becomes unreasonably costly as image scale increases.

Previous approaches:

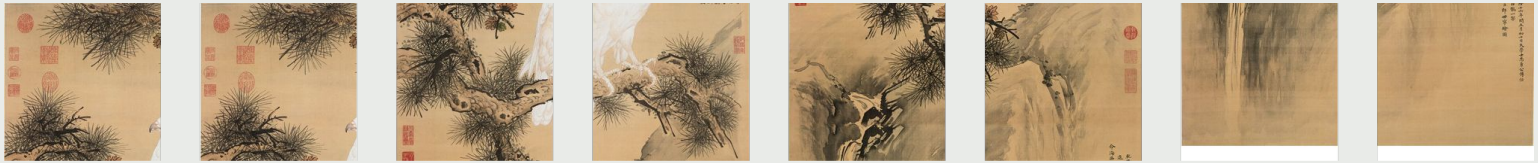
- Attention only for nearby pixels, keeping convolution.
  - Parmer et al (2018)
- Same as above, but no convolution.
  - Hu et al (2019)
  - Ramachandran et al (2019)
  - Zhao et al (2020)
- Sparse Transformers: scaleable approximation of attention.
  - Child et al (2019)
- Apply attention only within specific blocks or along axes.
  - Weissenborn et al (2019)
  - Ho et al (2019)
  - Wang et al (2020)
- **Apply attention to 2x2 patches of the image.**
  - Cordonnier et al (2020)
- iGPT: Full self-attention, but on a scaled down, grayscale image.
  - Chen et al (2020)

ViT Implementation: Patches

白鶻圖 — 郎世寧





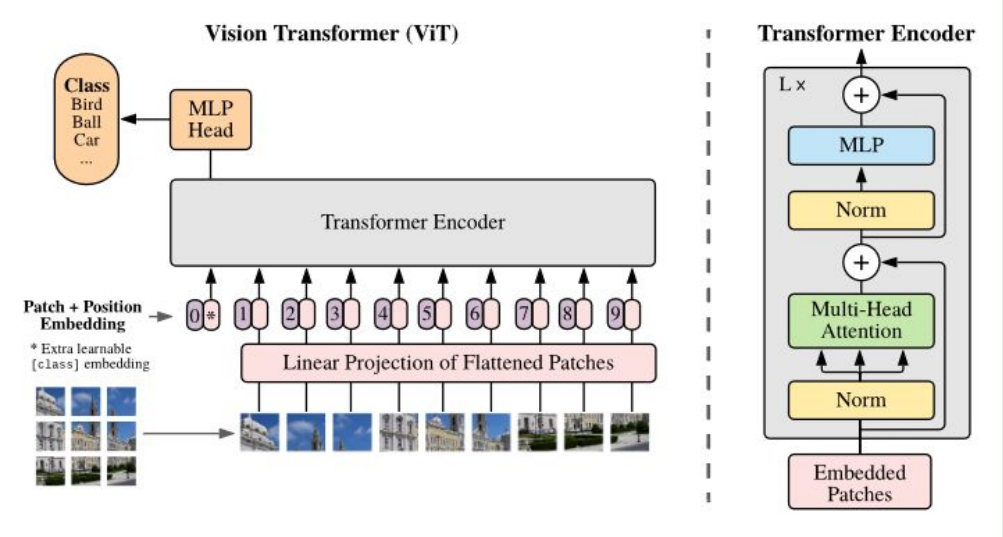


ViT Implementation: Non-Patch Details

Position Embedding: Added to the image patches is a position embedding, which is a value corresponding to the patch's position in the sequence (left to right, top to bottom). It only has 1D information; 2D positional embeddings did not improve performance.

Classification Token: The transformer outputs a "classification token" indicating what kind of image it thinks the input is.

Pre-training vs. Fine-tuning: Pre-training is done on lower-resolution images than fine-tuning.



Performance versus Previous SotA: ViT and SotA Variants

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

ViT Variants

Nomenclature: ViT-L/16 - Large variant of the Vision Transformer, with a 16x16px patch size.

Training: Pre-trained using Adam, fine-tuned with SGD.

State-of-the-Art Variants

Big Transfer (BiT): ResNet model introduced by Kolesnikov et al (2020).

Noisy Student: EfficientNet model introduced by Xie et al (2020).

Metrics

TPUv3-core-days: The total amount of time across all TPUv3 cores used to train the model, expressed as days.

Benchmark Score: How the models scored against a benchmark.

Performance versus Previous SotA: Benchmark Results

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Table 2: Comparison with state of the art on popular image classification benchmarks. We report mean and standard deviation of the accuracies, averaged over three fine-tuning runs. Vision Transformer models pre-trained on the JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less computational resources to pre-train. ViT pre-trained on the smaller public ImageNet-21k dataset performs well too. *Slightly improved 88.5% result reported in Touvron et al. (2020).

Performance versus Previous SotA: VTAB Results

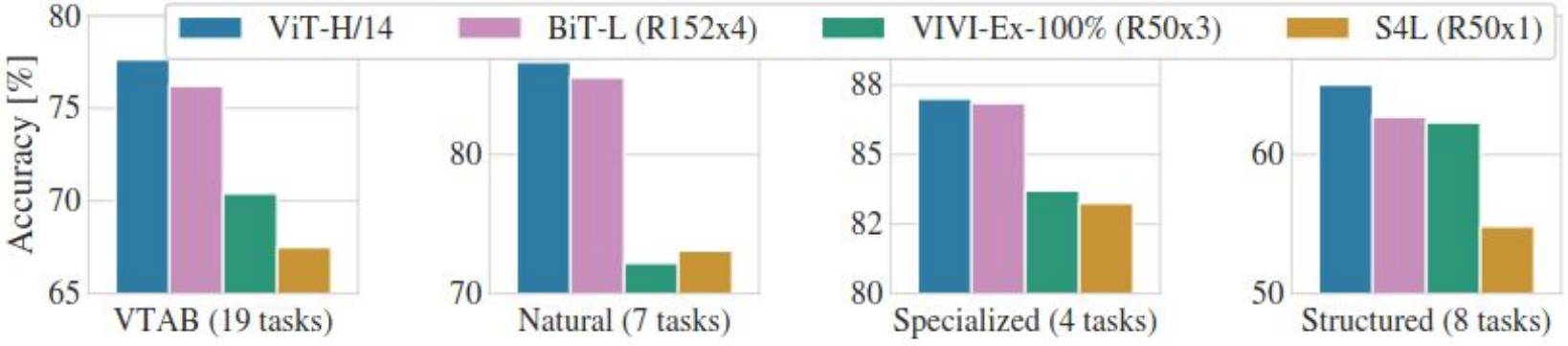
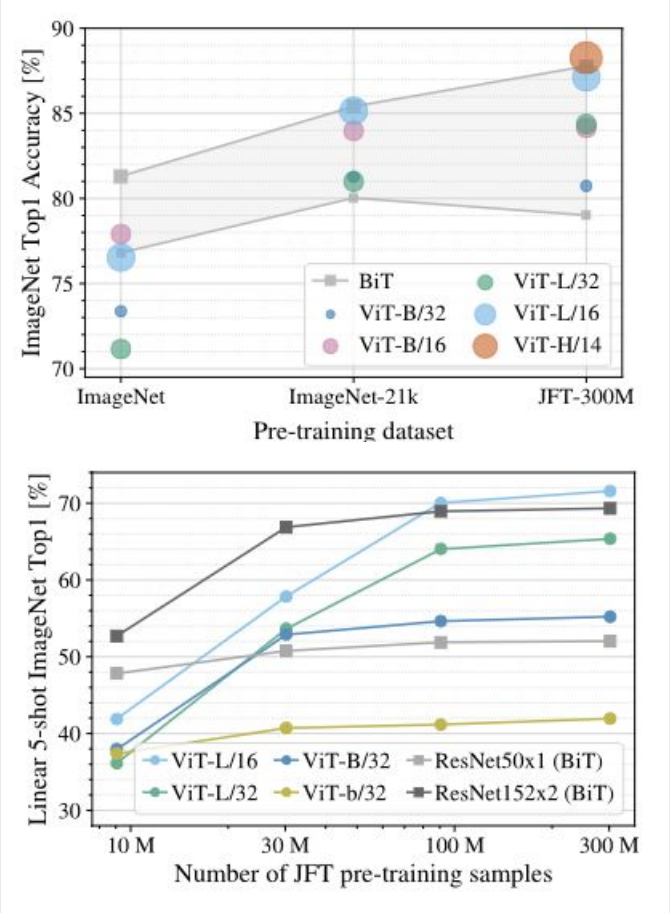


Figure 2: Breakdown of VTAB performance in *Natural*, *Specialized*, and *Structured* task groups.

Performance versus Previous SotA: Importance of Training Data Volume

ViT **performs worse** than BiT when trained on **small datasets**.

However, ViT **performs better** than BiT when trained on **large datasets**.



Legacy

This paper has 21,320 citations. For comparison, *Attention Is All You Need* has 78,010.

Models based on ViT:

- Meta's DINo (https://huggingface.co/docs/transformers/model_doc/dinov3)
- OpenAI's CLIP (https://arxiv.org/pdf/2609.14447)
- Google's SigLIP (https://arxiv.org/pdf/2502.14786)

Top right: performance of the fined-tuned GPT-3 model. Bottom right: performance with a verifier. Bottom left: sample of the GSM8K.